

M3 — Trade Data Core

Module Specification

Trading Journal App · v1.0 · July 2026 · Status: ready to break into tickets

Field	Value
Module ID	M3
Track / owner	Data In track — one owner, end to end (the same owner as M1 and M2)
Depends on	M0 Foundation & Shared Contracts · M1 Identity (trader's timezone) · M2 Broker Integration (RawTrade) · M5 Trade Logging (planned risk, edits — consumed, not blocking)
Blocks	M5, M6, M7 and M10 cannot show a single meaningful number until M3 exists — they all read Trade and AggregateSnapshot, never RawTrade directly.
Scope in	Normalising RawTrade into one canonical Trade record, grouping partial fills, session detection, P&L and R-multiple computation, outlier detection, incremental aggregation, recalculation on edit or timezone change, internal inspection tooling
Scope out	Fetching data from a broker (M2) · strategy definitions and rule-checking (M4) · the trader-facing logging and planning UI (M5) · dashboards and visualisation (M6) · any coaching or narrative interpretation (M7)

Purpose

M3 is the engine no trader ever sees directly. It takes the raw, unopinionated facts M2 hands it and turns them into one canonical trade with a session, a P&L, an R-multiple where possible, and an outlier flag — then keeps every downstream summary correct as new trades arrive and old ones get corrected. M3 never talks to a broker and never asks a trader a question; it only produces facts and derived numbers, never an opinion — that's M7's job.

Components in this specification

1. Stories
2. Data flow and schema
3. UI screens
4. Detailed description and business logic
5. Flowcharts
6. Test plan

7. Quality benchmarks
8. Security constraints
9. Error handling
10. External dependencies
11. Performance and scalability
12. Module relationships
13. Data policies (EU and US)
14. Documentation

1. Stories

Fourteen stories cover the module. M3 has no primary trader-facing screen, so most stories describe a trader-visible outcome produced entirely by automation, plus the handful of internal-tooling stories that keep the engine debuggable.

ID	Story	Acceptance criteria	Pri
US-01	As a trader, I want every trade to show a clear win/loss and P&L automatically, with no maths on my part.	Trade.pnl computed on ingestion from open/close price, volume, side, fees and swap, correct for the instrument's contract size.	Must
US-02	As a trader, I want my trades grouped by the session I actually traded in, using my own timezone.	Trade.session derived from open_time converted into the trader's M1 timezone against configured session windows (Figure 3).	Must
US-03	As a trader, I want partial fills and scaled entries treated as one trade, not several.	Executions matching the grouping rule become one TradeGroup with a single net P&L and volume.	Must
US-04	As a trader, I want unusually large wins or losses flagged so they don't distort my stats.	is_outlier computed against the trader's own trailing R-multiple distribution; the trader can see why a trade was flagged.	Must
US-05	As a trader, I want to see my risk-reward ratio per trade, not just profit or loss.	r_multiple computed when a planned risk is available from M5; left empty and clearly labelled otherwise — never fabricated.	Must
US-06	As a trader, I want daily, weekly and monthly summaries that load instantly, even with years of history.	AggregateSnapshot rollups are precomputed and updated incrementally, never recalculated from scratch on dashboard load.	Must
US-07	As a trader, I want my stats to update automatically if I fix a mistake on a trade.	Editing a trade (via M5) triggers a RecalculationJob scoped to just the affected aggregates.	Must
US-08	As a trader, I want my performance broken down by symbol, so I know which instruments work for me.	AggregateSnapshot supports symbol as a grouping dimension alongside day, week, month	Should

		and session.	
US-09	As a trader, I want yesterday's trades to already show clean numbers this morning, not "still processing."	Normalisation latency meets the target in Section 7 — p95 under the stated threshold from sync to a ready Trade.	Must
US-10	As a trader, I want a manually entered trade to have exactly the same stats as a synced one.	The normalisation pipeline treats M2's live, CSV and manual sources identically past ingestion.	Must
US-11	As a trader, I want changing my account's timezone to update all my past session labels, not just future trades.	A timezone change triggers a full session-relabel RecalculationJob across the trader's entire history.	Must
US-12	As a trader with an open position, I want it visible in my journal as "open," not silently ignored.	Trade.status = open is supported and shown; open trades are excluded from win-rate and closed-trade aggregates.	Should
US-13	As a support engineer, I want to see exactly how a trade's numbers were computed, to help a trader who reports something looks wrong.	The internal trade inspector (M3-A01) shows every input and intermediate value behind a Trade's final numbers.	Must
US-14	As a trader, I want a broker-corrected trade to update cleanly, not create a duplicate.	A correction matching an existing dedup key updates the Trade in place and writes a TradeEdit entry, never a second Trade row.	Must

Ticket-splitting note

US-01, US-02 and US-05 are the smallest shippable slice — turn a RawTrade into a Trade with a session, a P&L, and an R-multiple where possible. Build and verify that against real M2 data before adding grouping, outliers, or aggregation.

2. Data flow and schema

Six entities. M3 is the sole owner of every one of them; M2 never writes a Trade, and no module other than M3 is allowed to compute P&L, session, or an outlier flag.

Entity	Purpose	Key fields
Trade	The canonical, normalised trade — M3's entire reason for existing.	id · user_id · raw_trade_id (nullable) · symbol · side · volume · open_time (UTC) · close_time (UTC) · open_price · close_price · fees · swap · pnl · r_multiple (nullable) · session · status (open closed) · is_outlier · outlier_reason · source · calc_version · created_at · updated_at
TradeGroup	Bundles related executions (partial fills, scale-in/out) into one logical trade.	id · user_id · primary_trade_id · member_trade_ids[] · grouping_reason · net_volume · net_pnl
TradeEdit	Audit history of every correction — manual, broker-sourced, or support-initiated.	id · trade_id · field · old_value · new_value · source (trader broker support) · reason · edited_at
AggregateSnapshot	Precomputed rollups that make dashboards fast.	id · user_id · dimension (day week month session symbol) · dimension_key · trade_count · win_count · total_pnl · win_rate · avg_r · computed_at
SessionWindow	Configurable Asian/London/New York/overlap time boundaries (Section 4.3, Figure 3).	id · label · utc_start · utc_end · priority · updated_at
RecalculationJob	One tracked run of recomputation, of any of the four kinds in Figure 2.	id · trigger_type · scope (single_trade date_range full_account) · status · started_at · finished_at · trades_affected · error_code

2.1 How data moves

Normalisation. A new RawTrade (from M2) or a manual entry triggers the pipeline in Figure 1: grouping check, session detection, P&L, R-multiple where a planned risk exists, and an outlier score — ending with an incremental patch to the affected AggregateSnapshot rows and a trade.normalized event.

Correction. A trader edit (via M5), a broker-sourced correction (via M2), or a support-initiated backfill all write a TradeEdit entry and trigger a RecalculationJob scoped to what actually changed — one trade or a date range, never the whole account unless explicitly requested.

Timezone change. M1 emits a signal when a trader's timezone changes. This is the one trigger that touches a trader's entire history: every historical Trade's session is re-derived and every AggregateSnapshot for that trader is rebuilt from scratch, not patched (Figure 2).

Reading. M5, M6, M7 and M10 all read Trade and AggregateSnapshot — never RawTrade, and never compute P&L or session logic themselves. This is the boundary that keeps “what happened” (M2) and “what it means” (M3) from blurring together.

The one rule that keeps this clean

M3 never talks to a broker and never asks a trader a question — its only inputs are RawTrade from M2 and edits routed through M5, M2, or support tooling. Everything it produces is a fact or a derived number, never an opinion. Opinions are M7's job.

3. UI screens

M3 has no trader-facing screen — every one of its outputs is consumed inside M5, M6, M7 and M10's own screens. What it does need is internal tooling, because a calculation engine a support engineer can't inspect is a calculation engine nobody can trust. Five screens, built as a working mockup in M3_UI_Mockups.html — open it in any browser.

ID	Screen	States built
M3-A01	Trade inspector	Found · Not found · Loading
M3-A02	Recalculation job monitor	Job list · Job detail · Empty
M3-A03	Session window configuration	Default · Editing · Overlap error
M3-A04	Outlier detection QA sample	Default · Rated
M3-A05	Aggregate snapshot debugger	No drift · Drift found · Running

3.1 Rules that apply to every screen

- Staff-only. None of these five screens are reachable from the trader-facing app under any condition — access is gated the same way as any other internal tool (Section 8).
- Read-mostly. Only M3-A02 (resume/retry a job) and M3-A03 (edit a global session window) allow a write; everything else is inspection.
- Every number shown traces back to a source — a RawTrade, an M5 plan, a prior AggregateSnapshot — never a bare figure with no lineage.
- A screen that could affect every trader at once (M3-A03) says so before the action is taken, not after.

4. Detailed description and business logic

The rules below are the module's real content. Where a rule looks unusually strict, the reasoning is stated — those are the ones that get quietly dropped otherwise.

4.1 Normalisation

Rule	Decision
One canonical shape	Every Trade has the same fields regardless of whether it came from MT5, cTrader, a CSV, or manual entry. Source-specific quirks are resolved during normalisation, not carried forward.
Unit and fee handling	P&L accounts for the instrument's contract size, plus fees and swap, using per-instrument reference data. A missing contract size fails the trade explicitly (Section 9) rather than silently computing a wrong number.
Idempotency	Re-normalising the same RawTrade (e.g. after a re-sync)

	must produce the same Trade, not a duplicate — this is what makes M2's dedup and M3's normalisation safe to overlap.
Open positions	A trade without a close_time is stored with status = open and normalised as far as possible (session, partial P&L where the broker reports it) but excluded from closed-trade aggregates until it closes.

4.2 Trade grouping

Rule	Decision
Grouping trigger	Same symbol, same account, executions overlapping or within a short contiguous time window, consistent with a scale-in/scale-out or partial fill rather than two independent trades.
Net figures	A TradeGroup's net P&L and volume are computed from its members, not summed naively — a scaled exit's partial closes must net against the remaining open volume correctly.
Reversibility	Grouping is a M3 judgement, not a broker fact — if it's ever wrong, the correction path is the same TradeEdit/RecalculationJob mechanism as any other fix, not a special case.

4.3 Session detection

Rule	Decision
Timezone source	Always the trader's M1 timezone — never a fixed default, never the broker server's timezone. This is the single hardest dependency M3 has on another module.
Window storage	SessionWindow boundaries are stored in UTC and are global, not per-trader — session labels are derived per-trader at read/compute time, never stored against a fixed clock (see M3-A03).
Overlap priority	Windows have an explicit priority order (Figure 3) so an overlap window can win over the two plain windows it geometrically sits inside, deterministically.
Off-session trades	A trade outside every configured window is labelled off-session/gap, not dropped and not force-fit into the nearest window — a wrong label would be worse than an honest “doesn't fit.”

DST	Session windows are evaluated against the trader's IANA timezone (from M1), so daylight-saving transitions are handled by the timezone library, never by a hand-maintained offset.
-----	--

4.4 Metrics computation

Rule	Decision
P&L formula	(open_price – close_price, sign-adjusted for side) × volume × contract_size, minus fees and swap. Documented and versioned (calc_version) so a formula fix is traceable against every trade computed under the old version.
R-multiple	Computed only when a planned risk is available (from an M5 plan linked to the trade). Never estimated or backfilled from the trade's own outcome — that would silently turn a risk measure into a restatement of the result.
Null is a real value	A trade with no R-multiple shows as “not available,” never as 0 or blank — OR and “no plan” are different facts and must never be visually confused.

4.5 Outlier detection

Rule	Decision
Method	A trade's R-multiple divided by the trader's own trailing R-multiple standard deviation (90-day window). This is personal to each trader — one trader's normal-sized win is another's outlier.
Threshold	2.5× by default, a starting constant reviewed quarterly against the QA sample in M3-A04, not hand-tuned per trader.
Cold start	A trader with under 10 closed trades has no meaningful trailing stdev — outlier detection is simply not run, rather than run against a noisy sample of one or two trades.
Not a gate	Flagging is instant and fully automatic; nothing about a trader's own dashboard waits on staff review (Section 3, M3-A04).

4.6 Aggregation and recalculation

Rule	Decision
------	----------

Default: incremental	A normal ingestion or a scoped edit patches only the AggregateSnapshot rows the change actually affects — day, week, month, session and symbol buckets touched by that one trade.
Full rebuild: rare and explicit	Only a timezone change (or an M3-A03 global session-window edit) triggers a full rebuild, because both can move which bucket every historical trade belongs to.
Drift is checked, not assumed	M3-A05 exists because incremental patching is the one place a silent bug would be invisible to a trader — their dashboard would just be quietly wrong.

5. Flowcharts

Three diagrams: how a raw trade becomes a finished Trade, what triggers a recalculation and how far it reaches, and how a single trade gets assigned to a session. Each is followed by the reasoning behind its branches.

5.1 Normalisation pipeline

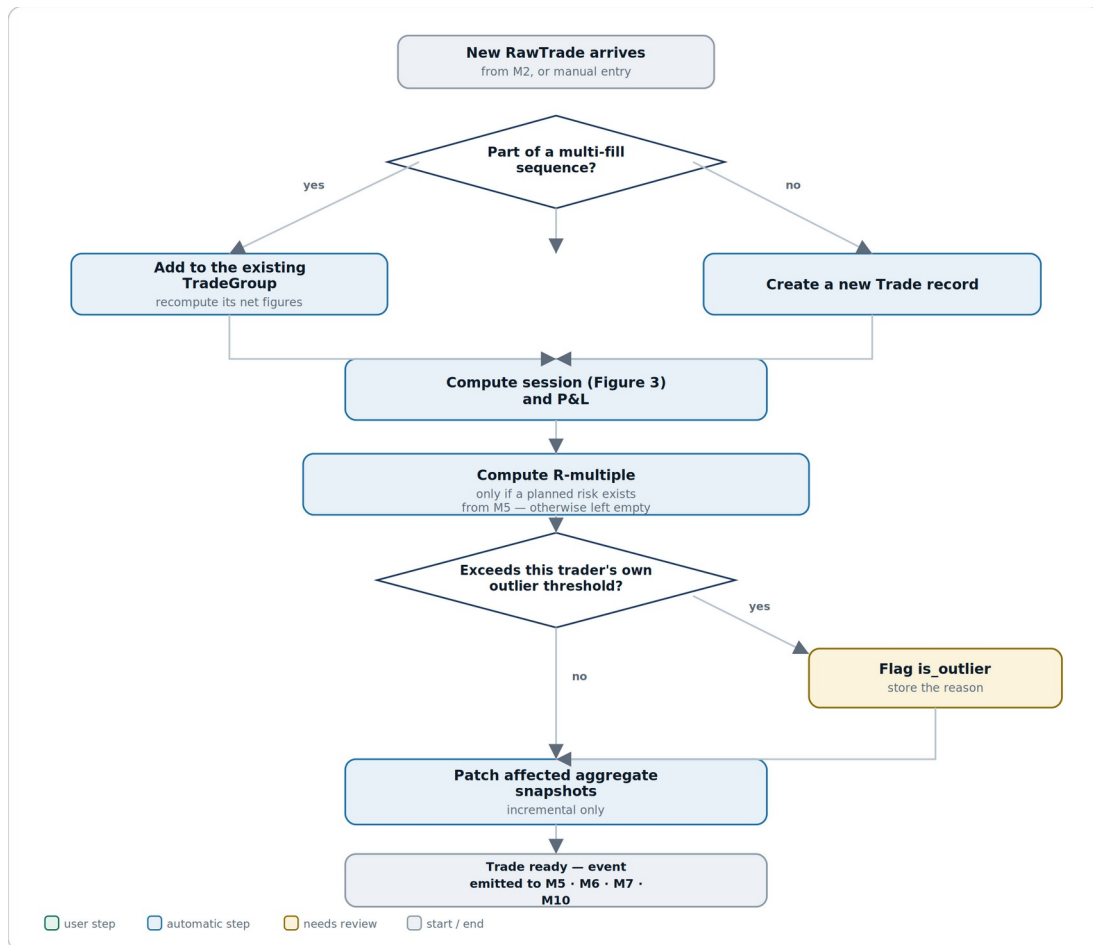


Figure 1 — from a RawTrade to a Trade ready for M5, M6, M7 and M10.

Reading it. The pipeline is almost entirely linear — one decision about grouping near the top, one about outlier status near the bottom, everything else a straight sequence. That's deliberate: normalisation should be boring and predictable, because the moment it isn't, a trader's numbers stop being trustworthy.

Why R-multiple sits before the outlier check, not after. Outlier scoring is defined in terms of R-multiple (Section 4.5), so it has to exist first. A trade with no planned risk simply can't be scored for R-based outliers — it still gets a P&L, just no outlier flag from this method.

5.2 What triggers a recalculation

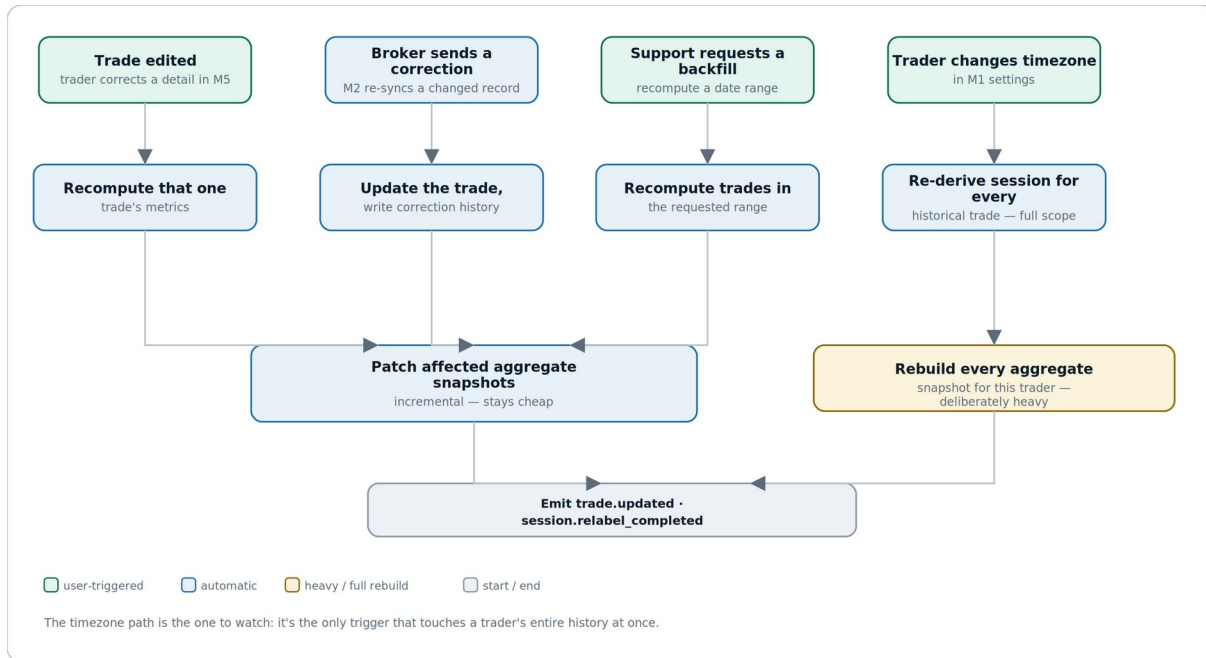


Figure 2 — four triggers, and the difference between a cheap patch and a full rebuild.

Reading it. Three of the four triggers stay narrow in scope — one trade, or a bounded date range — and patch only the aggregates they touch. The fourth, a timezone change, is structurally different: it can move every single historical trade into a different session bucket at once, so nothing short of a full rebuild is correct.

Why the two outcomes are kept visibly separate. Collapsing “patch” and “rebuild” into one generic “recalculate” step would hide the cost difference from anyone reading this later — a patch is milliseconds, a rebuild on a large account is minutes (Section 11). Keeping them as different boxes is what makes M3-A02's “Scope” column meaningful.

5.3 Session assignment

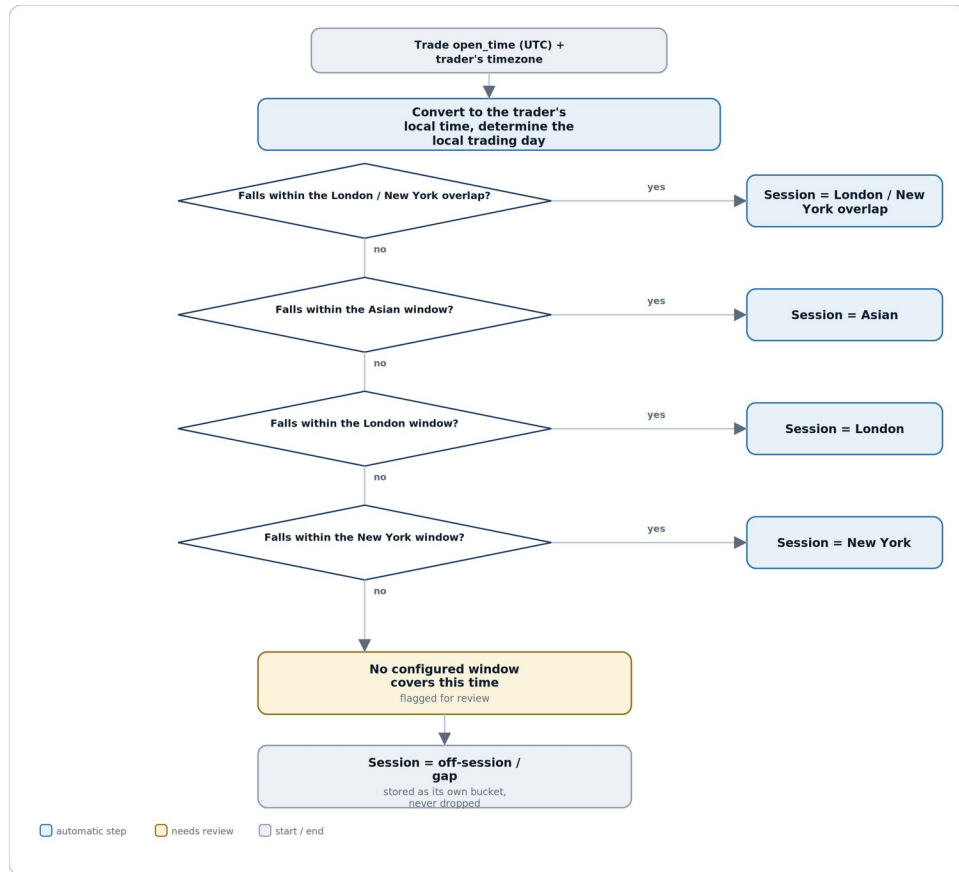


Figure 3 — how one trade's timestamp becomes a session label.

Reading it. This is a checklist, not a branching tree — each window is checked in a fixed priority order and the first match wins. The overlap window is checked first on purpose: it geometrically sits inside both the London and New York windows, so checking it last would mean it could never win.

Why “off-session” is a real, named outcome. A trader can trade outside every configured window — a crypto perp at 3am, for instance. Forcing that trade into the nearest session would quietly corrupt that session's statistics; labelling it honestly keeps every other session's numbers clean.

6. Test plan

A calculation engine fails silently by default — a wrong P&L formula doesn't crash anything, it just produces a confident, wrong number. Correctness here is tested at the level of exact figures, not just pass/fail behaviour.

Layer	What it covers	Key cases
Unit	Pure calculation logic with no I/O.	P&L formula across long/short and every fee combination · R-multiple only computed when planned risk exists · outlier score maths · session-window priority resolution
Integration	The module against its own contract and store.	Full RawTrade → Trade pipeline · TradeGroup net figures with 3+ partial fills · incremental AggregateSnapshot patch matches a full recompute
Contract	What M3 promises other modules.	trade.normalized / trade.updated fire with the agreed shape · Trade fields match exactly what M5/M6/M7/M10 expect, with no undocumented nulls
Recalculation	Every trigger in Figure 2, and its scope.	Single-trade edit patches only its own aggregates · timezone change rebuilds every session label and every snapshot · a failed job resumes from its last successful point (M3-A02)
Regression	Formula changes don't silently rewrite history.	A calc_version bump does not retroactively alter already-computed Trades unless a backfill is explicitly requested
Drift	Snapshot correctness over time.	M3-A05 comparison passes on a large seeded account after a mix of edits, timezone changes, and partial-fill grouping

6.1 Manual QA checklist

#	Manual QA checklist item
1	A trade with 4 partial fills nets to one TradeGroup with the correct combined P&L.
2	Changing a trader's timezone in M1 visibly re-labels session badges on historical trades, not just new ones.
3	A trade with no linked M5 plan shows R-multiple as “not

	available,” never as 0.
4	An outlier flag disappears if the underlying trade is edited to no longer qualify.
5	The trade inspector's computation trace (M3-A01) matches the actual stored values exactly, for at least 5 sampled trades.
6	A failed recalculation job (M3-A02) resumes from its last successful trade, not from the start.
7	Editing a session window (M3-A03) is blocked with a clear error when it creates an unresolvable overlap.
8	The snapshot debugger (M3-A05) reports zero drift on a freshly rebuilt account.
9	An open (unclosed) trade appears in the journal but is excluded from win-rate aggregates.
10	A broker-sourced correction updates the existing Trade in place — no duplicate row, and a TradeEdit entry is written.

6.2 Test data

- A seeded account with 5,000+ closed trades across all four sessions, for aggregation and performance testing.
- A cluster of partial fills designed to test grouping edge cases (scale-in then scale-out, overlapping but not identical timing).
- Trades straddling a DST transition in at least two timezones, to catch session-boundary bugs.
- A trade intentionally missing instrument reference data, to exercise the CALC-002 error path (Section 9).
- An account with both broker-synced and manually entered trades, to confirm parity (US-10).

7. Quality benchmarks

Numbers, not adjectives. Anything unmeasurable is not a benchmark.

Metric	Target	How it is measured
Normalisation latency	New RawTrade becomes a ready Trade within 3 s, p95	Time from RawTrade.imported_at to Trade.created_at
Incremental patch latency	A single-trade recalculation completes within 1 s, p95	RecalculationJob duration, scope = single_trade
Full account rebuild	10,000 trades re-derived and every snapshot rebuilt within 8 minutes	RecalculationJob duration, scope = full_account
Dashboard query latency	Any AggregateSnapshot read returns within 200 ms, p95	Server timing on snapshot reads — never a live recompute on the read path
Snapshot accuracy	0 unresolved drift findings older than 24 hours	M3-A05 scheduled comparison run, alerts on any open drift
Outlier flagging precision	≥ 85% of QA-sampled flags rated “correctly flagged”	Quarterly review of M3-A04 staff ratings
Grouping correctness	0 incorrectly merged or incorrectly split TradeGroups in QA regression	Automated regression suite against seeded partial-fill fixtures

Why snapshot accuracy is the number that matters most

Every other module trusts M3's numbers without re-deriving them — M6's dashboard, M7's coaching, M10's streaks all read an AggregateSnapshot and take it as fact. A silent drift here doesn't cause an error anywhere; it just makes every downstream feature confidently wrong at the same time.

8. Security constraints

M3 holds the derived truth behind a trader's entire financial track record, plus five staff-only tools that can see and, in two cases, change it. Everything below is a requirement, not a recommendation.

Area	Constraint
Internal tool access	M3-A01 through M3-A05 are reachable only by authenticated staff accounts with an explicit internal-tools role — never by the trader-facing app's own auth, even for a trader's own data.
Read vs write separation	Only M3-A02 (resume/retry) and M3-A03 (edit a session window) can write anything. The other three screens are

	strictly read-only, enforced at the API layer, not just hidden in the UI.
No direct database edits	Every correction, whatever its source, goes through the TradeEdit + RecalculationJob path — there is no supported way to hand-edit a Trade's stored numbers outside that mechanism.
Session-window changes are audited	Every M3-A03 edit is logged with the staff account, the old and new values, and the RecalculationJob it triggered — this is a global, trader-affecting action and must be fully attributable.
Calc versioning	Every Trade records the calc_version used to produce it, so a formula bug fix is distinguishable from a value that was always correct under an older formula.
No secrets held here	M3 holds no broker credentials and no authentication material — that risk lives entirely in M1 and M2. M3's exposure is data integrity, not credential theft.
Logging	Trade inspector lookups (M3-A01) and session-window edits (M3-A03) are logged with the requesting staff identity, to support later audit of who viewed or changed what.

9. Error handling

Every error has a code for logs, a defined system behaviour, and — since M3's audience is mostly other systems and staff, not traders directly — a clear description of what shows up downstream.

Code	Scenario	System behaviour	What shows downstream
CALC-001	RawTrade missing a required field (price, volume, timestamp)	Normalisation fails for that trade only; queued for retry	Trade does not appear until resolved; visible as a pending item in M3-A02
CALC-002	No instrument reference data (contract size) for the trade's symbol	Normalisation blocked, job marked failed with this code	Support sees the exact missing symbol in M3-A02's job detail (see the worked example in Section 3)
CALC-003	P&L formula produces a non-finite or absurd result (e.g. divide-by-zero on a zero-volume record)	Trade flagged for manual review, not silently stored	Excluded from aggregates until reviewed; visible in M3-A01 with a review flag
GRP-001	Ambiguous grouping — two plausible TradeGroups for the same execution	Defaults to the safer choice (treat as a separate trade), logged for review	No trader-visible error; a support engineer can re-group manually via a backfill
SES-001	Trade's local time matches no configured SessionWindow	Session = off-session/gap, not blocked	Trade is fully usable; shown with an "off-session" label rather than a wrong one (Figure 3)
SES-002	Overlapping SessionWindow definitions with equal priority (M3-A03)	Save rejected before any recalculation is queued	Staff sees the exact overlapping ranges and must resolve priority or narrow the range
JOB-001	A RecalculationJob fails partway through a multi-trade scope	Job marked failed, progress preserved, resumable from the last successful trade	Visible in M3-A02 with exact progress and the specific failure
JOB-002	A full-account rebuild is requested while one is already running for that trader	Second request rejected, not queued twice	Requester is told a rebuild is already in progress with its current progress
DRIFT-001	M3-A05 finds a snapshot that doesn't match a live recompute	Flagged, not auto-corrected silently	Surfaced in M3-A05 with the exact delta; requires an explicit rebuild action

Why silent auto-correction is deliberately avoided

DRIFT-001 and GRP-001 could technically be auto-resolved without asking anyone. They aren't, because a system

that quietly fixes its own financial calculations is one nobody can audit later — every correction here leaves a visible trace, even the ones a machine could have safely made alone.

10. External dependencies

M3 is almost entirely internal compute — its dependency list is short compared to M1 or M2, and that contrast is itself worth noting.

Dependency	Used for	If it fails	Fallback
M2 Broker Integration	The RawTrade facts M3 normalises	No new trades to process — M3 simply has nothing new to do	Not a real failure mode for M3 itself; M3 stays correct on whatever it already has
M1 Identity	The trader's timezone for session detection	Session detection cannot run correctly	M1 guarantees timezone is never null (see M1 spec, Section 4.3) — this dependency is a hard one, not a degrade-gracefully one
M5 Trade Logging	Planned risk values for R-multiple, and trader-initiated edits	R-multiple stays unavailable for affected trades; edits can't be applied	P&L and session still compute fully without it — R-multiple is the only field that depends on M5
Instrument reference data	Contract size and pip/point conventions per symbol	Normalisation blocked for that symbol (CALC-002)	Existing Trades already computed are unaffected; only new trades on that symbol queue until data is added
Job scheduler / queue	Running RecalculationJobs and the scheduled drift check (M3-A05)	Recalculations queue but don't run; snapshots may drift undetected	Alert on-call; queued jobs resume once the scheduler recovers, per Section 9's JOB-001 behaviour

Why this list is short on purpose

M3 is a pure transformation layer — it reads facts, applies deterministic rules, and writes derived facts. It has no third-party API, no OAuth flow, and no file uploads. That's a direct consequence of the M2/M3 split: all the messy, unreliable, third-party surface area was deliberately pushed into M2, leaving M3 free to be boring, predictable, and fast.

11. Performance and scalability

Concern	Position
Scale assumption for v1	Up to 50,000 trades per active trader and 20,000 active traders. Nothing in M3 should need re-architecting below

	that.
Hottest path	Normalising a newly synced trade (Figure 1). It must scale horizontally per trade, independent of how large any one trader's history is.
Deliberate slow path	A full-account rebuild after a timezone change (Figure 2). Expected to take minutes on a large account and runs as a background job with visible progress (M3-A02), never inline.
Read path is always precomputed	M5, M6, M7 and M10 read AggregateSnapshot rows, never trigger a live recompute on request — a slow dashboard load would mean this rule was broken somewhere.
What breaks first	The RecalculationJob queue under a burst of timezone changes or session-window edits happening close together — both are full-account-scope operations and should be throttled per-tenant so one large account can't starve everyone else's normal ingestion.
Growth triggers	Revisit design at 200,000 trades for a single trader, sustained normalisation latency above the Section 7 target, or full-account rebuilds regularly exceeding 15 minutes.

12. Module relationships

This is the contract surface. Once agreed, other modules can build against it with mocks while M3 is still being written.

Direction	Module	What crosses the boundary
Exposes	M5 Trade Logging	Trade (with computed P&L, session, R-multiple where available) for a trade the trader is viewing or editing; accepts planned-risk attachment and edits back.
Exposes	M6 Dashboard	AggregateSnapshot rows by day, week, month, session and symbol — M6 never computes a statistic itself, only renders what M3 already computed.
Exposes	M7 AI Coach	Trade history and aggregates as the factual substrate for coaching — M3 supplies facts, M7 supplies interpretation, and that boundary is not M7's to cross.
Exposes	M10 Gamification	Win-rate and streak-relevant aggregates, read-only.
Consumes	M2 Broker Integration	RawTrade rows and the broker.trade_imported / broker.backfill_completed events that trigger normalisation.
Consumes	M1 Identity	The trader's timezone, and the user.timezone_changed event that triggers a full rebuild.
Consumes	M5 Trade Logging	Planned-risk values attached to a trade, and trader-initiated edits — both arrive as write requests into M3's TradeEdit path, not as direct table writes.
Consumes	M0 Foundation	Shared data model, design system, and error contract.
Emits	—	trade.normalized · trade.updated · trade.outlier_flagged · aggregate.updated · session.relabel_completed
Triggers	M5, M6, M7, M10	On trade.normalized or trade.updated, each downstream module re-

		evaluates whatever it owns — a logged entry's display, an open dashboard, a coaching insight, a streak.
--	--	---

The dependency to watch

M3 cannot detect a session without a timezone from M1, and cannot compute an R-multiple without a planned risk from M5 — both are hard, not optional, dependencies for those specific fields. Agree the exact Trade field names, types, and null-ability (especially r_multiple's nullability) in M0 before M5 or M6 build against a guess. This is the same class of handoff M1 flagged for M3's timezone dependency — it is the reason M0 exists.

13. Data policies (EU and US)

M3 doesn't collect anything new — every field it holds is derived from data M1 and M2 already gathered with the trader's consent. Its obligation is to propagate the same rights correctly, not to establish new ones.

13.1 What we hold and why

Data	Classification	Lawful basis (GDPR)
Trade (computed P&L, session, R-multiple, outlier flag)	Personal data; commercially sensitive financial data, derived from M2's raw facts	Contract — the core analysis the trader signed up for
TradeGroup and TradeEdit history	Personal data — explains how a trader's numbers were derived and corrected	Contract; also supports accuracy obligations (GDPR Art. 5(1)(d))
AggregateSnapshot	Personal data, derived and summarised	Contract
Internal tool access logs (M3-A01/A03)	Personal data about staff, and indirectly about the trader record viewed	Legitimate interests — security and accountability

13.2 GDPR (EU / UK)

Obligation	How M3 meets it
Lawful basis (Art. 6)	Contract — Trade and AggregateSnapshot are the direct output of the service the trader signed up for.
Accuracy (Art. 5(1)(d))	TradeEdit history and calc_version exist specifically so an incorrect derived value can be identified, corrected, and explained — this is an accuracy obligation as much as a debugging convenience.
Data minimisation (Art. 5(1)(c))	M3 derives only what the product's stated features need — P&L, session, R-multiple, outlier status — not speculative scores with no defined use.
Right of access / portability (Art. 15/20)	Trade and derived metrics are included in M1's account-wide export; M3 does not maintain a separate export path.
Erasure (Art. 17)	On user.deletion_requested from M1, M3 must erase or anonymise every Trade, TradeGroup, TradeEdit and AggregateSnapshot row for that user within the same 30-day grace period M1 defines.
Automated decision-making (Art. 22)	Outlier flagging and metric computation inform what a trader sees about their own trades; they do not produce any automated decision with legal or similarly significant

	effect on the trader, so Art. 22's stricter regime does not apply. Worth re-checking if M7 or M10 ever build a decision on top of these flags that could change that answer.
Security of processing (Art. 32)	Internal tool access is restricted to staff with an explicit role (Section 8); no trader-facing credential grants access to another trader's Trade data.

13.3 United States (CCPA / CPRA and state law)

Requirement	How M3 meets it
Sensitive personal information	Derived financial performance data is treated with the same sensitivity as the trade history it comes from (per M2's classification) — no lesser standard just because it's computed rather than raw.
Right to know / access	Same account-wide export as GDPR access, via M1 — no separate M3 export.
Right to delete	Same erasure flow as GDPR Art. 17, propagated within the account-level grace period.
Right to correct	A trader corrects a trade through M5's edit flow, which M3 honours via TradeEdit — there is no separate “correct my stats” mechanism, because stats are always derived, never directly editable.
Not investment advice	Outlier flags and R-multiples describe what already happened; they are not phrased or delivered as a recommendation. That framing discipline matters most in M7, but the underlying data M3 produces must stay equally neutral.

13.4 Retention schedule

Data	Retention	Reason
Trade, TradeGroup	Life of account + 30-day grace (matches M1/M2)	Core product value — a trader's history is not a log to be pruned early
TradeEdit history	Same as the Trade it explains	An edit history detached from its trade is meaningless and shouldn't outlive or underlive it
AggregateSnapshot	Same as the underlying Trades — deleted alongside them	Derived data has no purpose once its source is erased
RecalculationJob logs	12 months	Operational history and incident

		investigation
Internal tool access logs (M3-A01/A03)	12 months	Staff accountability and audit

Two things not to get wrong

First: erasure must genuinely cascade — if M1 erases a user but M3 still holds their Trade rows or AggregateSnapshot, the product has not honoured the deletion it promised, exactly as flagged in M2's spec.

Second: a calc_version change must never retroactively alter a stored historical Trade without an explicit, logged backfill — silently reinterpreting the past is the one failure mode that would make every number in the app untrustworthy at once.

14. Documentation

Documentation is part of the module, not an afterthought once the code works.

Deliverable	Contents
Module README	What M3 does, how to run the normalisation pipeline locally, how to seed a realistic test account with partial fills and multiple sessions.
Contract / API reference	Every field on Trade and AggregateSnapshot, generated from the contract agreed in Section 12 and M0, including exact nullability rules.
Decision log (ADRs)	Initial entries: why R-multiple is never estimated without a planned risk; why grouping defaults to the safer non-merge on ambiguity (GRP-001); why a timezone change forces a full rebuild instead of a smarter partial one; why calc_version exists.
Runbook	What to do when normalisation backs up, when a RecalculationJob queue grows unexpectedly, when M3-A05 reports drift, and how to safely re-run a failed backfill.
Data inventory & retention register	The tables in Section 13, maintained as living documents rather than a one-off.
Formula reference	The exact, versioned P&L, R-multiple, and outlier-score formulas — the document a support engineer opens before touching M3-A01.
UI mockups	M3_UI_Mockups.html — all five internal screens with every state.

14.1 Definition of done

M3 is done when all of the following are true:

- Every story meets its acceptance criteria.
- The contract in Section 12 is published, and at least one other module (M5 or M6) has built against it — even if only with mocks.
- All five internal screens are built with every state and are gated to staff access only.
- Every error in Section 9 is reachable and produces the described system behaviour.
- A full timezone-change rebuild and a single-trade edit both work end to end, at the correct scope.
- The snapshot debugger (M3-A05) reports zero drift on a freshly built, heavily-edited test account.
- Benchmarks in Section 7 are measured, not assumed.
- README, ADRs, runbook and the formula reference are written; the data inventory is filled in.
- Deletion propagation is verified end to end from M1 through to M3's own tables.
- Demoted to the other two owners, with the Trade contract walked through explicitly.

End of M3 specification — v1.0. Same 14-component structure as M1 and M2; the third reference point as the pattern continues across M4–M12.